

Session 2018 – 2019 (Midterm)

1. **What are the Three main purposes of an operating system? We have stressed the need for an operating system to make efficient use of the computing hardware. When is it appropriate for the operating system to forsake this principle and to “waste” resources? Why is such a system not really wasteful?**

The three main purposes are:

- To provide an environment for a computer user to execute programs on computer hardware in a convenient and efficient manner.
- To allocate the separate resources of the computer as needed to perform the required tasks. The allocation process should be as fair and efficient as possible.
- As a control program, it serves **two major functions**: (1) Supervision of the execution of user programs to prevent errors and improper use of the computer, and (2) Management of the operation and control of I/O devices.

Single-user systems should maximize the use of the system for the user. GUI might “waste” CPU cycles, but it optimizes the user’s interaction with the system.

2. **Identify several advantages and several disadvantages of open-source operating systems. Identify the types of people who would find each aspect to be an advantage or a disadvantage. Describe some of the challenges of designing operating systems for mobile devices compared with designing operating systems for traditional PCs.**

Open-source operating systems have the advantages of having **many people working on them, many people debugging them, ease of access and distribution, and rapid update cycles**. Further, for **students and programmers**, there is certainly an advantage to being able to view and modify the source code. Typically open-source operating systems are free for some forms of use, usually just requiring payment for support services. Commercial operating system companies usually do not like the competition that open-source operating systems bring because these features are difficult to compete against. Some open-source operating systems do not offer paid support programs. Some companies avoid open-source projects because they need paid support so that they have some entity to hold accountable if there is a problem or they need help fixing an issue. Finally, some complain that a lack of discipline in the coding of open-source operating systems means that **backward- compatibility** is lacking making upgrades difficult and that the frequent release cycle exacerbates these issues by forcing users to upgrade frequently.

The greatest challenges in designing mobile operating systems include:

- Less storage capacity means the operating system must manage memory carefully.
- The operating system must also manage power consumption carefully.
- Less processing power plus fewer processors mean the operating system must carefully apportion processors to applications.

3. What is the purpose of system calls? List five services provided by an operating system, and explain how each creates convenience for users. In which cases would it be impossible for user-level programs to provide these services? Explain your answer.

System calls allow user-level processes to request services of the operating system.

Five services provided by an OS:

1. **Program execution:** The operating system loads the contents (or sections) of a file into memory and begins its execution. A user-level program could not be trusted to properly allocate CPU time.
2. **I/O operations:** It is necessary to communicate with disks, tapes, and other devices at a very low level. The user needs only specify the device and the operation to perform on it, and the system converts that request into device- or controller-specific commands. User-level programs cannot be trusted to access only devices they should have access to and to access them only when they are otherwise unused.
3. **File-system manipulation:** There are many details in file creation, deletion, allocation, and naming that users should not have to perform. Blocks of disk space are used by files and must be tracked. Deleting a file requires removing the name file information and freeing the allocated blocks. Protections must also be checked to assure proper file access. User programs could neither ensure adherence to protection methods nor be trusted to allocate only free blocks and deallocate blocks on file deletion.
4. **Communications.** Message passing between systems requires messages to be turned into packets of information, sent to the network controller, transmitted across a communications medium, and reassembled by the destination system. Packet ordering and data correction must take place. Again, user programs might not coordinate access to the network device, or they might receive packets destined for other processes.
5. **Error detection.** Error detection occurs at both the hardware and software levels. At the hardware level, all data transfers must be inspected to ensure that data have not been corrupted in transit. All data on media must be checked to be sure they have not changed since they were written to the media. At the software level, media must be checked for data consistency— for instance, whether the number of allocated and unallocated blocks of storage matches the total number on the device. There, errors are frequently processed independently (for instance, the corruption of data on a disk), so there must be a global program (the operating system) that handles all types of errors. Also, when errors are processed by the operating system, processes need not contain code to catch and correct all the errors possible on a system.

4. Contrast and compare an application programming interface (API) and an application binary interface (ABI) Explain why Java programs running on Android systems do not use the standard Java API and virtual machine.

comparing API & ABI

	 API	 ABI
abstraction level	higher (software)	lower (hardware)
built upon	objects, primitive data types	registers
scope	source code, languages	machine code, code binary
example	system calls, functions	AAPCS64, System V

It is because the **standard API** and virtual machine are designed for desktop and server systems, not mobile devices. Google developed a separate API and virtual machine for mobile devices.

Session 2018 – 2019 (Final)

1.

- a) What are the Three main purposes of an operating system? How does the distinction between kernel mode and user mode function as a rudimentary form of protection?

First part of the answer already noted.

The distinction between kernel mode and user mode provides a **rudimentary form of protection** in the following manner. *Certain instructions can be executed only when the CPU is in kernel mode. Similarly, hardware devices can be accessed only when the program is in kernel mode, and interrupts can be enabled or disabled only when the CPU is in kernel mode.* Consequently, the CPU has minimal capability when executing in user mode, thereby enforcing the protection of critical resources.

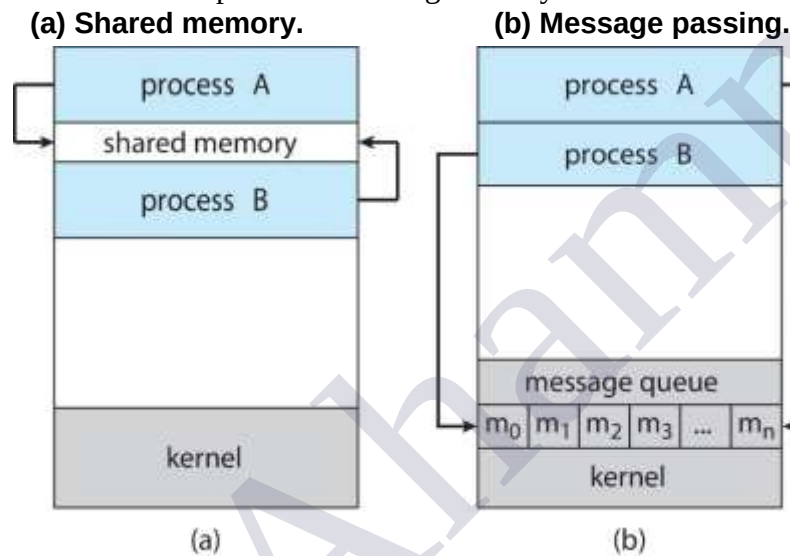
- b) We have stressed the need for an operating system to make efficient use of the computing hardware. When is it appropriate for the operating system to forsake this principle and to “waste” resources? Why is such a system not really wasteful?

Single-user systems should maximize the use of the system for the user. GUI might “waste” CPU cycles, but it optimizes the user’s interaction with the system.

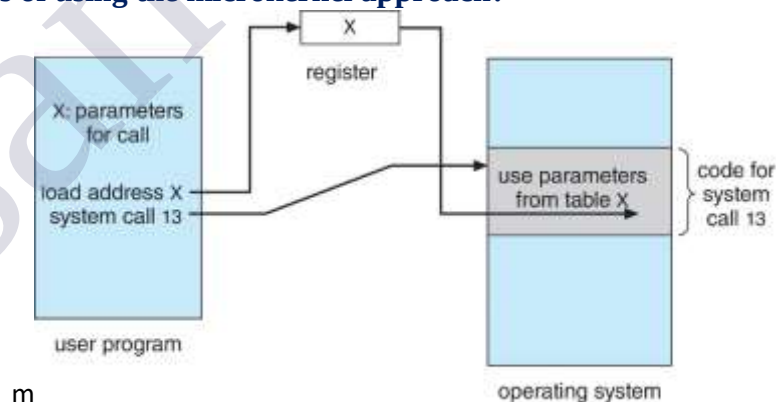
- c) What is the purpose of system calls? What are the two models of inter-process communication? What are the strengths and weaknesses of the two approaches?

System calls allow user-level processes to request services of the operating system.

The two models of inter-process communication are the **message-passing model** and the **shared-memory model**. Message passing is useful for exchanging smaller amounts of data because no conflicts need to be avoided. It is also easier to implement than shared memory for inter-computer communication. Shared memory allows maximum speed and convenience of communication. Since it can be done at memory transfer speeds when it takes place within a computer. However, this method compromises protection and synchronization between the processes sharing memory.



- d) Describe three general methods for passing parameters to the operating system. What is the main advantage of the microkernel approach to system design? How do user programs and system services interact in a microkernel architecture? What are the disadvantages of using the microkernel approach?

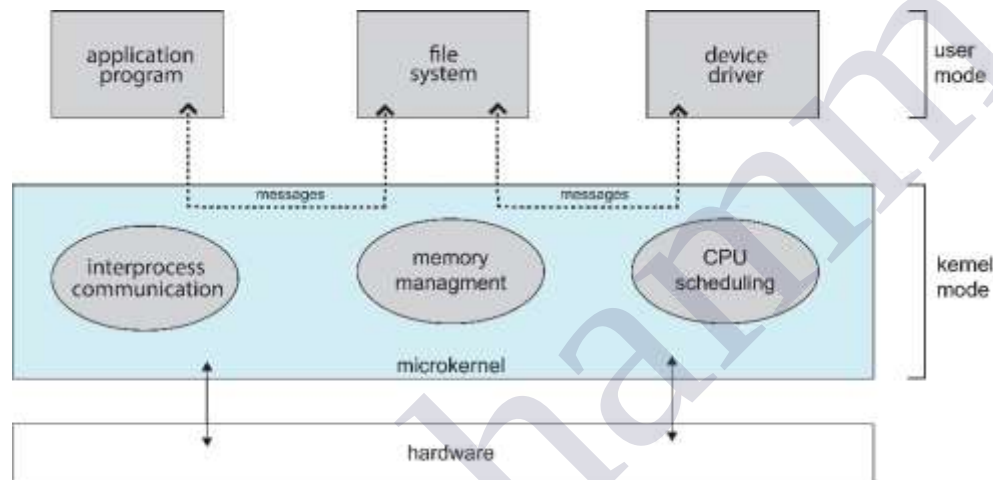


- 1) Pass parameters in registers.
- 2) Registers pass starting addresses of blocks of parameters.
- 3) Parameters can be placed, or pushed, onto the stack by the program, and popped off the stack by the operating system.

Benefits typically include the following:

- Adding a new service does not require modifying the kernel.
- It is more secure as more operations are done in user mode than in kernel mode.
- A simpler kernel design and functionality typically results in a more reliable operating system.

User programs and system services interact in a micro-kernel architecture by using **inter-process communication mechanisms such as messaging**. These messages are conveyed by the operating system.



The primary disadvantages of the microkernel architecture are the overheads associated with inter-process communication and the frequent use of the operating system's messaging functions in order to enable the user process and the system service to interact with each other.

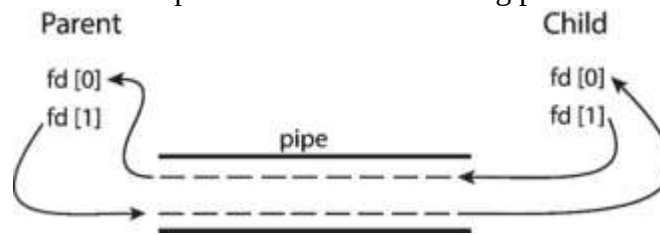
2.

- Define process. Give an example of a situation in which ordinary pipes are more suitable than named pipes and an example of a situation in which named pipes are more suitable than ordinary pipes.

A **process** refers to a set of coordinated steps or activities aimed at achieving a specific result. It typically involves the transformation of inputs into outputs through a series of actions.

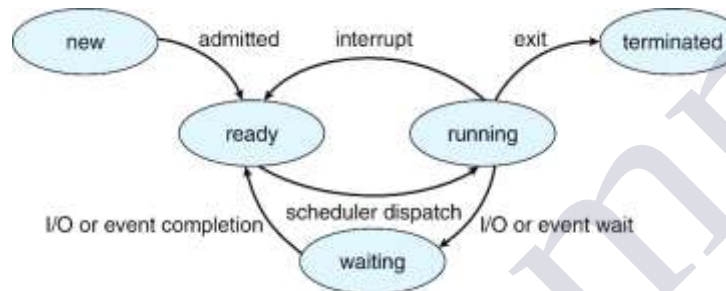
Simple communication works well with ordinary pipes. For example, assume we have a process that counts characters in a file. An ordinary pipe can be used where the producer writes the file to the pipe and the consumer reads the files and counts the number of characters in the file. Next, **for an example where named pipes are more suitable**, consider the situation where several processes may write messages to a log. When processes wish to write a message to the log, they write it to the named pipe. A server reads the messages from the named pipe and writes them to the log file.

Require parent-child relationship between communicating processes



- b. Draw and describe the diagram of process state. Describe the actions taken by a kernel to context switch between processes.

Diagram of process state:



This diagram represents the **lifecycle of a process** in an operating system. Here are the states and transitions:

1. **New:** The process is being created
2. **Running:** Instructions are being executed
3. **Waiting:** The process is waiting for some event to occur
4. **Ready:** The process is waiting to be assigned to a processor
5. **Terminated:** The process has finished execution.

During a context switch between kernel-level threads, the kernel performs these actions:

1. **Save the current thread's state:** The kernel saves the CPU registers, program counter, and other details of the current thread so it can resume later.
2. **Choose the next thread to run:** The kernel picks another thread from the ready queue (a list of threads waiting to run).
3. **Load the new thread's state:** The kernel loads the saved state (registers, program counter, etc.) of the new thread.
4. **Switch to the new thread:** The CPU now runs the new thread, starting from where it left off.

This process ensures smooth switching between threads without losing their progress.

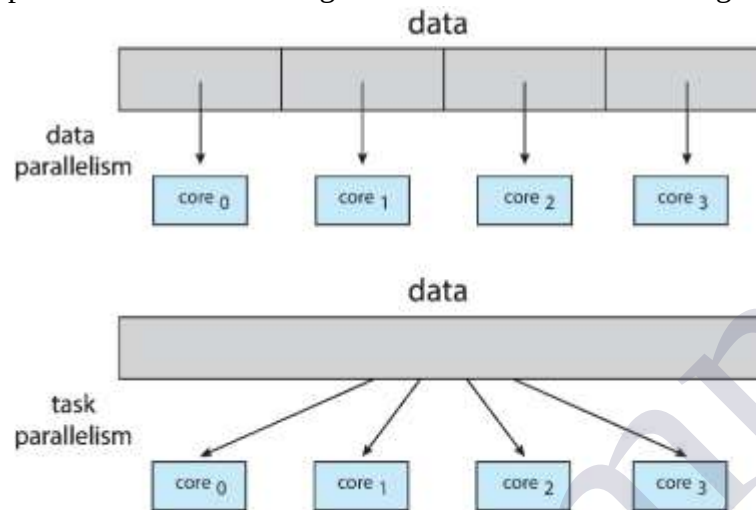
- c. **Define data and task parallelism. Describe about different multithreading models.**

Data Parallelism: Multiple processors perform the same task on different parts of data simultaneously.

Example: Applying a filter to different sections of an image at the same time.

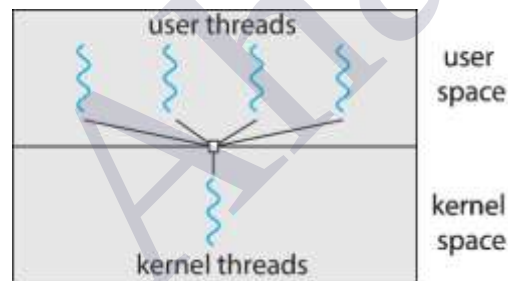
Task Parallelism: Multiple processors perform different tasks on the same or different data simultaneously.

Example: One processor resizes an image while another converts it to grayscale.



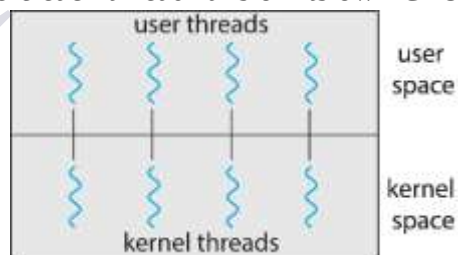
Many-to-One: Multiple user threads are mapped to one kernel thread. Only one thread runs at a time, so no parallelism.

Example: A single-threaded Python program where many tasks exist, but only one runs at a time.



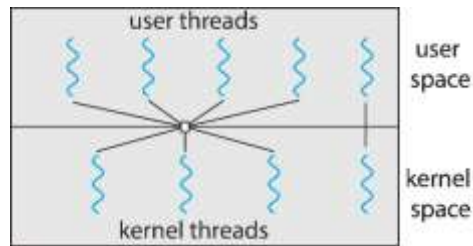
One-to-One: Each user thread has its own kernel thread. True parallelism is possible but can use a lot of resources.

Example: Java threads where each thread runs on its own CPU core.



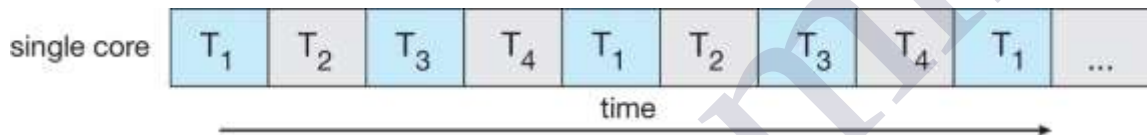
Many-to-Many: Many user threads are mapped to a few kernel threads. Allows flexibility and better resource use.

Example: Windows or Linux systems where multiple tasks are balanced across fewer kernel threads.

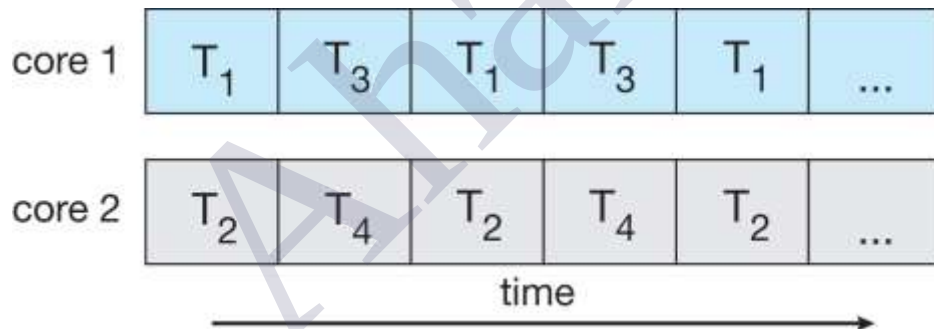


- d. Define thread. Show the concurrent execution on a single-core system and parallel multicore system. Using Amdahl's Law, calculate the speedup gain of an application that has a 60 percent parallel component for (i) two processing cores and (ii) four processing cores.

A thread is a sequence of programmed instructions that the CPU can execute independently. It shares the process's resources (like memory, file handles, etc.) but runs its own code and maintains its own execution state (registers, stack, program counter).



Parallel multicore system.



Amdahl's Law is used to predict the theoretical maximum speedup of a program based on the proportion of the program that can be parallelized and the number of processing cores (or processors) used. It is expressed as:

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

Where:

- $S(N)$ is the speedup of the system with N processing cores.
- P is the parallelizable portion of the application (in decimal form).
- N is the number of processing cores.

Given that:

- $P = 0.60$ (60% of the application is parallel).
- The remaining $1 - P = 0.40$ (40% is not parallelizable).

We will calculate the speedup for two scenarios:

1. For 2 cores ($N = 2$)
2. For 4 cores ($N = 4$)

1. Speedup for 2 cores ($N = 2$):

$$S(2) = \frac{1}{(1 - 0.60) + \frac{0.60}{2}}$$

$$S(2) = \frac{1}{0.40 + \frac{0.60}{2}}$$

$$S(2) = \frac{1}{0.40 + 0.30}$$

$$S(2) = \frac{1}{0.70}$$

$$S(2) \approx 1.43$$

So, the speedup gain for 2 cores is approximately **1.43**.

2. Speedup for 4 cores ($N = 4$):

$$S(4) = \frac{1}{(1 - 0.60) + \frac{0.60}{4}}$$

$$S(4) = \frac{1}{0.40 + \frac{0.60}{4}}$$

$$S(4) = \frac{1}{0.40 + 0.15}$$

$$S(4) = \frac{1}{0.55}$$

$$S(4) \approx 1.82$$

So, the speedup gain for 4 cores is approximately **1.82**.

3.

a) Consider a variant of the RR scheduling algorithm in which the entries in the ready queue are pointers to the PCBs.

I. What would be the effect of putting two pointers to the same process in the ready queue?

Imagine you have 3 people waiting in line for ice cream, but one person has their name written twice on the list. The person with their name twice will get ice cream more often than the others.

In the same way, if a process has two pointers in the ready queue, it will run twice as often as other processes, giving it more time on the CPU.

II. What would be two major advantages and two disadvantages of this scheme?

1. Advantages:

- **More CPU time for important tasks:** If a process is important, giving it more pointers makes sure it gets extra CPU time.
 - Example: You want a video game to run smoothly, so you give it extra turns in the queue.
- **Simple way to prioritize:** By adding extra pointers, you can quickly make one process run more than others.
 - Example: If a file download is slow, you can give it extra turns to speed it up.

2. Disadvantages:

- **Unfair to others:** Other processes may have to wait too long if one process has more pointers.
 - Example: If the same person keeps getting ice cream twice, others may not get any in time.
- **Harder to manage:** Keeping track of multiple pointers for one process makes the system more complicated.
 - Example: It's like having two lists of names for the same person—confusing to manage!

III. How would you modify the basic RR algorithm to achieve the same effect without the duplicate pointers?

Instead of giving a process multiple pointers, you can use **Weighted Round Robin (WRR)**. In WRR, each process gets a **weight** (a number that tells how many times it will run before moving to the next).

b) Suppose that the following processes arrive for execution at the times indicated. Each process will run for the amount of time listed. In answering the questions, use **non-preemptive scheduling**, and base all decisions on the information you have at the time the decision must be made.

Process	Arrival Time	Burst Time
P1	0.0	8
P2	0.4	4
P3	1.0	1

I. What is the average turnaround time for these processes with the FCFS scheduling algorithm?

- P1 runs first (0.0 – 8.0), so its TAT is 8.0.
- P2 waits for P1 (8.0 – 12.0), so its TAT is 11.6.
- P3 waits for both (12.0 – 13.0), so its TAT is 12.0.

Average turnaround for these processes: $(8 + (12 - 0.4) + (13 - 1)) / 3 = 10.53$

II. What is the average turnaround time for these processes with the SJF scheduling algorithm?

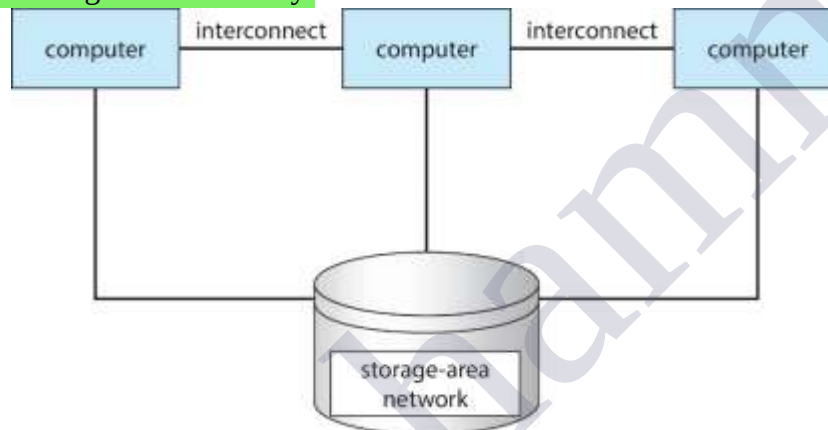
- Turnaround Time (TAT) = Completion Time - Arrival Time
- P1 TAT = 8.0 - 0.0 = 8.0
- P3 TAT = 9.0 - 1.0 = 8.0
- P2 TAT = 13.0 - 0.4 = 12.6

Average turnaround for these processes: $(8 + (9 - 1) + (13 - 0.4)) / 3 = 9.53$

Session 2017 – 2018 (Midterm)

1. How do clustered systems differ from multiprocessor systems? What is required for two machines belonging to a cluster to cooperate to provide a highly available service?

Clustered systems are typically constructed by combining multiple computers into a single system to perform a computational task distributed across the cluster. Multiprocessor systems on the other hand could be a single physical entity comprising multiple CPUs. A clustered system is less tightly coupled than a multiprocessor system. Clustered systems communicate using messages, while processors in a multiprocessor system could communicate using shared memory.



In order for two machines to provide a highly available service, the state of the two machines should be replicated and should be consistently updated. When one of the machines fails, the other could then take over the functionality of the failed machine.

2. Describe some distributed applications that would be appropriate for a peer-to-peer system (ch-01). The services and functions provided by an operating system can be divided into two main categories. Briefly describe the two categories, and discuss how they differ. (Ch-02)

Essentially anything that provides content, in addition to existing services such as file services, distributed directory services such as domain name services, and distributed e-mail services.

Operating system services can be grouped into two main categories:

1. User Services

These are the functions that help users interact with the computer easily. They include:

- **Running programs:** Starting and using applications.
- **File management:** Creating, deleting, and organizing files.
- **User interface:** Providing a way to interact with the computer, like through icons or commands.

Example: When we open a program by clicking its icon, that's a user service.

2. System Services

These work behind the scenes to keep the computer running smoothly. They include:

- **Managing resources:** Distributing CPU, memory, and storage to programs.
- **Security:** Controlling access to files and resources.
- **Device control:** Managing devices like keyboards and printers.

Example: When the system decides how much memory each program gets, that's a system service.

3. Describe three general methods for passing parameters to the operating system. What is the main advantage of the microkernel approach to system design? How do user programs and system services interact in a microkernel architecture? What are the disadvantages of using the microkernel approach?

Already noted.

Session 2017 – 2018 (Final)

1.

- a) Distinguish between the client-server and peer-to-peer models of distributed systems. Some early computers protected the operating system by placing it in a memory partition that could not be modified by either the user job or the operating system itself. Describe two difficulties that you think could arise with such a scheme.

client-server and peer-to-peer models:

Client/Server	Peer-To-Peer
Server has the control ability while clients don't	All computers have equal ability
Higher cabling cost	Cheaper cabling cost
It is used in small and large networks	Normally used in small networks with less than 10 computers
Easy to manage	Hard to manage
Install software only in the server while the clients share the software	Install software to every computer
One powerful computer acting as server	No server is needed

Placing the operating system in a protected memory partition could cause two main difficulties:

1. Limited Flexibility

If the operating system can't modify itself, updates, bug fixes, or improvements would be difficult or impossible without restarting the system or using a complicated process to unlock and modify the partition.

2. Resource Management Issues

The fixed memory partition may not be large enough as the system grows. This can cause problems when the operating system needs more memory to handle new features or complex tasks, leading to inefficiency or crashes.

- b) **What are the Three main purposes of an operating system? We have stressed the need for an operating system to make efficient use of the computing hardware. When is it appropriate for the operating system to forsake this principle and to “waste” resources? Why is such a system not really wasteful?**

Already noted.

- c) **What is the purpose of system calls? It is sometimes difficult to achieve a layered approach if two components of the operating system are dependent on each other. Identify a scenario in which it is unclear how to layer two system components that require tight coupling of their functionalities.**

System calls allow user-level processes to request services of the operating system

The virtual memory subsystem and the storage subsystem are typically tightly coupled and require a careful design in a layered system due to the following interactions. Many systems allow files to be mapped into the virtual memory space of an executing process. On the other hand, the virtual memory subsystem typically uses the storage system to provide the backing store for pages that do not currently reside in memory. Also, updates to the file system are sometimes buffered in physical memory before it is flushed to disk, thereby requiring careful coordination of the usage of memory between the virtual memory subsystem and the file system.

2.

- a) **Describe the actions taken by a kernel to context-switch between processes.**

- Saves the current process's state (registers, program counter, etc.).
- Updates the process control block (PCB) with the current process's status.
- Selects the next process to run.
- Loads the new process's state from its PCB.
- Switches the memory context to the new process.

What are the benefits and the disadvantages of each of the following? Consider both the system level and the programmer level.

I. Synchronous and asynchronous communication

➤ Synchronous:

- **Benefits:** Ensures both parties are in sync, making coordination easier.
- **Disadvantages:** Can cause delays as processes must wait for each other.

➤ **Asynchronous:**

- **Benefits:** Allows processes to work independently, improving efficiency.
- **Disadvantages:** Harder to manage since processes may be out of sync, requiring more complex error handling.

II. Automatic and explicit buffering

➤ **Automatic:**

- **Benefits:** Simplifies programming by managing buffer memory automatically.
- **Disadvantages:** Less control for the programmer, may lead to inefficiency.

➤ **Explicit:**

- **Benefits:** Provides the programmer full control over how data is stored.
- **Disadvantages:** More complex to implement, increasing the chance of errors.

III. Send by copy and send by reference

➤ **Send by Copy:**

- **Benefits:** Ensures data is not accidentally changed by the receiver.
- **Disadvantages:** Uses more memory and can slow down large data transfers.

➤ **Send by Reference:**

- **Benefits:** Faster and uses less memory since only a reference is passed.
- **Disadvantages:** Risk of unintended data changes by the receiver.

IV. Fixed-sized and variable-sized messages

➤ **Fixed-sized:**

- **Benefits:** Simpler to manage and more predictable in memory allocation.
- **Disadvantages:** Can waste space if the message is smaller than the fixed size.

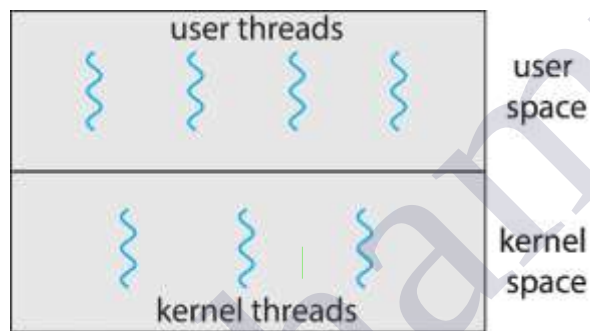
➤ **Variable-sized:**

- **Benefits:** More efficient use of memory as it adjusts to the message size.
- **Disadvantages:** Harder to manage and may require more complex handling.

- b) What are two differences between user-level threads and kernel-level threads? Under what circumstances does a multithreaded solution using multiple kernel threads provide better performance than a single-threaded solution on a single-processor system?

Difference between user-level threads and kernel-level threads in a table format:

Feature	User-Level Threads	Kernel-Level Threads
Managed by	Managed by the user-level library, not the OS	Managed directly by the operating system (OS)
Switching Speed	Switching between threads is faster	Switching between threads is slower



Multiple kernel threads provide better performance than a single-threaded solution when **one thread is blocked** (e.g., waiting for I/O), allowing another thread to continue executing. This can improve overall efficiency because the processor is not sitting idle during blocking operations.

- c) A system with **two dual-core processors** has **four processors available** for scheduling. A CPU-intensive application is running on this system. All input is performed at program start-up, when a single file must be opened. Similarly, all output is performed just before the program terminates, when the program results must be written to a single file. Between start-up and termination, the program is entirely CPU-bound. Your task is to improve the performance of this application by multithreading it. The application runs on a system that uses the one-to-one threading model (each user thread maps to a kernel thread).

- i. How many threads will you create to perform the input and output? Explain.

Since the file must be accessed sequentially, the work of reading and writing it cannot reasonably be parallelized, and **only a single thread** should be used for each of these tasks.

- ii. How many threads will you create for the CPU-intensive portion of the application? Explain.

The CPU-bound portion should be divided **evenly among the four processors**, so **four threads should be created for this part of the program**. Fewer than four would waste processor resources, while more threads would be unable to run simultaneously

3.

a) Explain the difference between preemptive and non-preemptive scheduling.

Preemptive Scheduling Vs Non Preemptive Scheduling		
Basis For Comparison	Preemptive Scheduling	Non Preemptive Scheduling
Basic	A Process can be suspended by other process	Once a process send to cpu that process can not be suspended.
Interrupt	Process can be interrupted in between	Process can not be interrupted till it terminates or switches to waiting state
Starvation	Possible	Possible
Flexibility	Flexible	Not flexible
Example	SRTF, LRTF, RR etc	FCFS, SJF, HRRN etc.
Cost	Preemptive Scheduling are cost associated	Non preemptive scheduling are not cost associated

Consider the following set of processes, with the length of the CPU burst given in milliseconds:

Process	Burst Time	Priority
P1	5	4
P2	3	1
P3	1	2
P4	7	2
P5	4	3

The processes are assumed to have arrived in the order P1, P2, P3, P4, P5, all at time 0.

- I. Draw four Gantt charts that illustrate the execution of these processes using the following scheduling algorithms: FCFS, SJF, non-preemptive priority (a larger priority number implies a higher priority), and RR (quantum = 2).

Order of arrival: P1, P2, P3, P4, P5 (all at time 0).

1. First-Come, First-Served (FCFS)

- Gantt Chart:

| P1 | P2 | P3 | P4 | P5 |
0 5 8 9 16 20

2. Shortest Job First (SJF)

- Gantt Chart:

| P3 | P2 | P5 | P1 | P4 |
0 1 4 8 13 20

3. Non-Preemptive Priority Scheduling

- Note: Higher priority is indicated by a larger number.
- Gantt Chart:

| P1 | P5 | P3 | P4 | P2 |
0 5 9 10 17 20

4. Round Robin (Quantum = 2)

- Gantt Chart:

| P1 | P2 | P3 | P4 | P5 | P1 | P2 | P4 | P5 | P1 | P4 | P4 |
0 2 4 5 7 9 11 12 14 16 17 19 20

II. What is the turnaround time of each process for each of the scheduling algorithms in part a?

Turnaround time = Completion time - Arrival time

	FCFS	SJF	Priority	RR
P1	5	13	5	17
P2	8	4	20	12
P3	9	1	10	5
P4	16	20	17	20
P5	20	8	9	16

III. What is the waiting time of each process for each of these scheduling algorithms?

Waiting time = Turnaround time - Burst time

	FCFS	SJF	Priority	RR
P1	0	8	0	12
P2	5	1	17	9
P3	8	0	9	4
P4	9	13	10	14
P5	16	4	5	12

IV. Which of the algorithms results in the minimum average waiting time (over all processes)?

Shortest Job First (SJF) has the minimum average waiting time.

b) Discuss how the following pairs of scheduling criteria conflict in certain settings.

I. CPU utilization and response time

- ✓ CPU utilization means keeping the CPU busy as much as possible.
- ✓ Response time is how quickly the system starts responding to a request.
- **Conflict:** If you focus on keeping the CPU busy (high CPU utilization), it might take longer to respond to new tasks (worse response time). For example, long-running tasks may be prioritized, making the system slower to respond to short tasks.

II. Average turnaround time and maximum waiting time.

- ✓ Average turnaround time is how long, on average, a process takes to finish.
- ✓ Maximum waiting time is the longest a process has to wait to start running.
- **Conflict:** Reducing average turnaround time by prioritizing quick tasks can cause some tasks to wait a long time, increasing their maximum wait. If you try to reduce the maximum wait, the average time to complete tasks may increase.

III. I/O device utilization and CPU utilization.

- ✓ I/O device utilization is how well devices like disks are kept busy.
- ✓ CPU utilization is how well the CPU is kept busy.
- **Conflict:** If the system is busy with I/O tasks (high I/O utilization), the CPU might sit idle, lowering CPU utilization. On the other hand, if the CPU is busy, I/O devices may not be fully used.

4.

a) Is it possible to have concurrency but not parallelism? Explain.

Yes, concurrency without parallelism is possible.

- ✓ Concurrency means tasks are making progress at the same time.
- ✓ Parallelism means tasks are running at the same time on different processors or cores.

Example: On a single-core CPU, only one task runs at a time, but it can switch between tasks quickly. This gives the feeling that both tasks are running together. This is concurrency but not parallelism because only one task runs at any moment.

b) Explain why Java programs running on Android systems do not use the standard Java API and virtual machine. Original versions of Apple's mobile iOS operating system provided no means of concurrent processing. Discuss three major complications that concurrent processing adds to an operating system.

It is because the standard API and virtual machine are designed for desktop and server systems, not mobile devices. Google developed a separate API and virtual machine for mobile devices.

Three complications concurrent processing adds to an operating system:

- 1) **Race Conditions:** When multiple tasks try to access and change the same data at the same time, results can become unpredictable.
- 2) **Deadlock:** Two or more tasks can get stuck, waiting on each other to finish, which causes the system to freeze.
- 3) **Resource Sharing:** Managing CPU, memory, and other resources among multiple tasks becomes harder, and one task might starve for resources while others use them up.

- c) The following processes are being scheduled using a preemptive, priority-based, round-robin scheduling algorithm.

Process	Priority	Burst time	Arrival time
P1	8	15	0
P2	3	20	0
P3	4	20	20
P4	4	20	25
P5	5	5	45
P6	5	15	55

Each process is assigned a numerical priority, with a higher number indicating a higher relative priority. The scheduler will execute the highest priority process. For processes with the same priority, a round-robin scheduler will be used with a time quantum of 10 units. If a process is preempted by a higher-priority process, the preempted process is placed at the end of the queue.

- I. Show the scheduling order of the processes using a Gantt chart.

P1	P2	P3	P4	P3	P5	P3	P6	P4	P2	
0	15	20	30	40	45	50	55	70	80	95

- II. What is the turnaround time for each process?

Turnaround Time = Completion Time - Arrival Time

Process	Arrival Time	Completion Time	Turnaround Time
P1	0	15	15
P2	0	95	95
P3	20	55	35
P4	25	80	55
P5	45	50	5
P6	55	70	15

- III. What is the waiting time for each process?

Waiting Time = Turnaround Time - Burst Time

Process	Burst Time	Turnaround Time	Waiting Time
P1	15	15	0
P2	20	95	75
P3	20	35	15
P4	20	55	35
P5	5	5	0
P6	15	15	0